



The Best Claude Code Setup for Bug Bounty Hunting

Turn Claude Code into a powerful bug bounty hunting assistant with MCP, custom skills, agents, tools and automated security workflows.

🔖🔖🔖🔖🔖

Published August 27, 2026

Free: No

[Read on Freedium](#) · [original](#)

Contents

Introduction	4
What Is Claude Code?	4
What is MCP	4
Step 1: Setting Up the Burp Suite MCP Server	5
Step 2: Traffic Collection and Reconnaissance	8
Step 3: The AI Bug Hunting Workflow	8
Phase 1: Analyzing and Organizing Traffic	9
Phase 2: Mapping the Attack Surface	9
Phase 3: Active Vulnerability Testing	10
Phase 4: Generating the Final Report	12
The One-Shot Prompt	12
Bonus: Bug Hunting Without Burp MCP	15
1. Analyze a Burp Raw Request	16
2. Analyze a JavaScript File	17
3. Testing a JavaScript File Directly from a URL	17
4. Analyze Multiple JS Files	18
5. Ask Claude to Investigate a Specific Vulnerability	18
6. Review Source Code	19
7. Turn Everything Into a Bug Bounty Report	20
The Full Shortcut	20
Claude Code Skills for Bug Hunting	22
Using Open-Source Bug Hunting Skills	22
Adding GitHub-Hosted Bug Hunting Skills	23
Creating Your Own Custom Bug Hunting Skills	23
Claude Skill Structure & Format	24
Example Prompts for Creating Custom Bug Hunting Skills	25

Open Redirect Skill	25
XSS Skill	26
Custom Methodology → Skill	26
Conclusion	28
Disclaimer	28

Introduction

Bug bounty hunting isn't just about finding vulnerabilities. It involves recon, analyzing JavaScript and HTTP requests, mapping the attack surface, testing and reporting. A lot of this work is repetitive and that's where Claude Code and MCP come in. In this guide, I'll show you how to set everything up from scratch, automate parts of the workflow and connect Claude Code directly to Burp Suite through MCP.

What Is Claude Code?

Claude Code is a terminal-based coding and agentic assistant from Anthropic. Instead of interacting with an AI only through a browser, you can run it directly from your terminal and allow it to work with files, commands and other tools. That makes it particularly interesting for security research because much of a bug bounty workflow already happens inside the terminal.

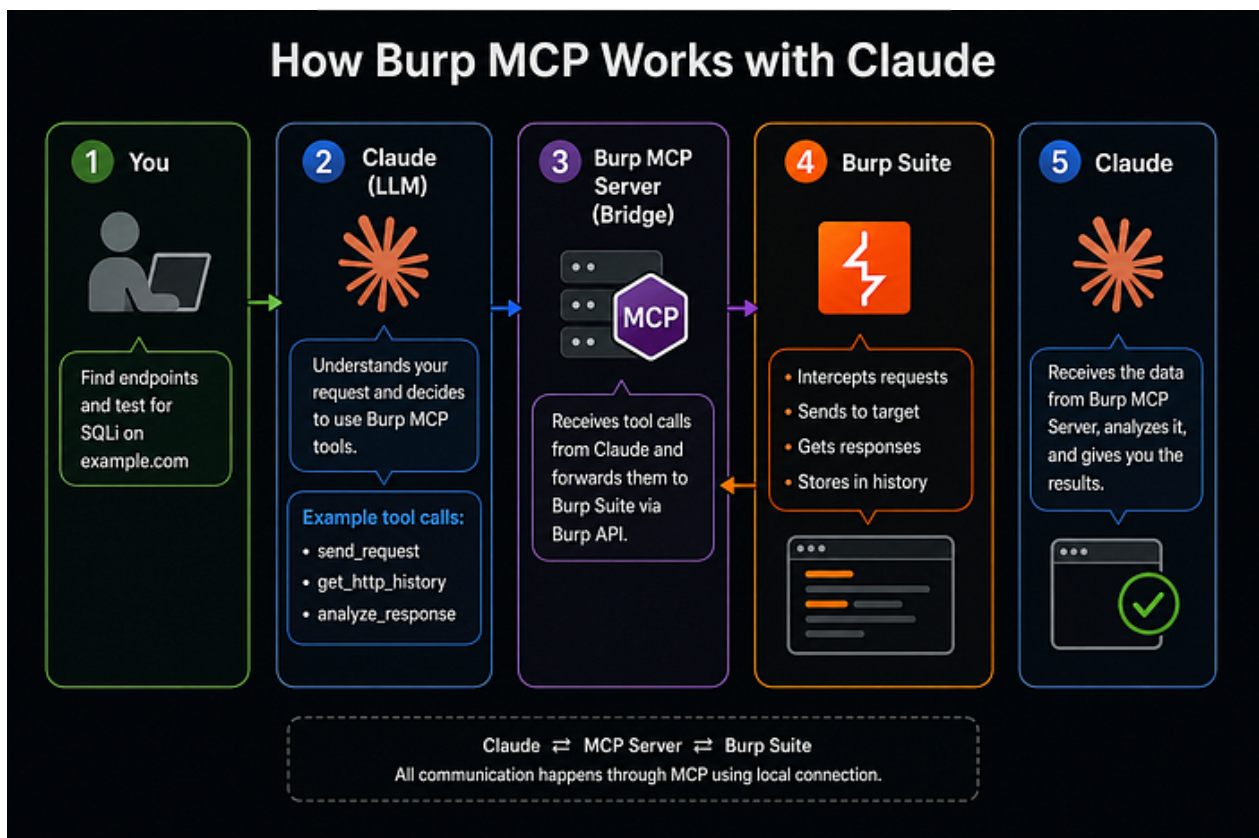
For example, you might have a directory containing:

```
target/  
├─ javascript/  
├─ requests/  
├─ urls.txt  
├─ endpoints.txt  
└─ notes/
```

Claude Code can work with that project context instead of forcing you to manually copy everything into a chat interface.

What is MCP

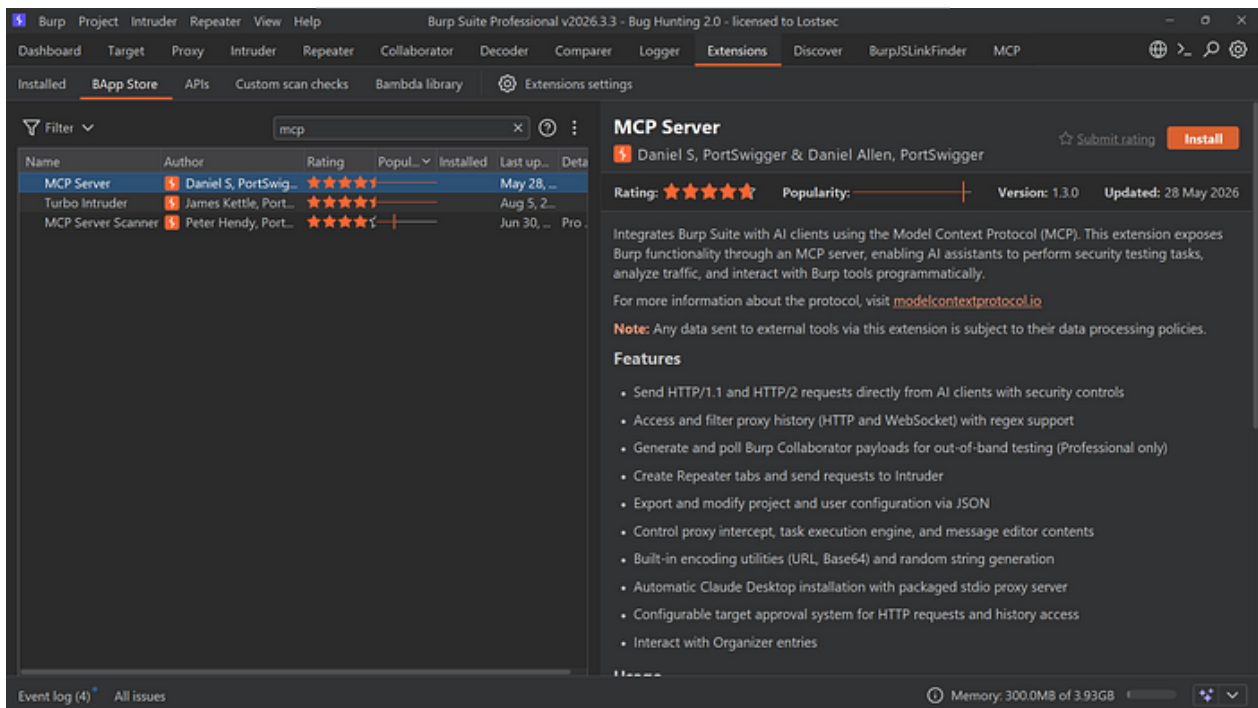
MCP stands for **Model Context Protocol**. In simple terms, it allows AI models like Claude to connect with external tools and actually interact with them. Think of Claude as the brain and MCP as the bridge. Without MCP, Claude can understand your instructions and analyze the information you provide, but it cannot directly interact with tools like Burp Suite. Once you connect the Burp Suite MCP server, Claude can access your Burp HTTP history and use the tools featured by Burp MCP to analyze and interact with captured traffic.



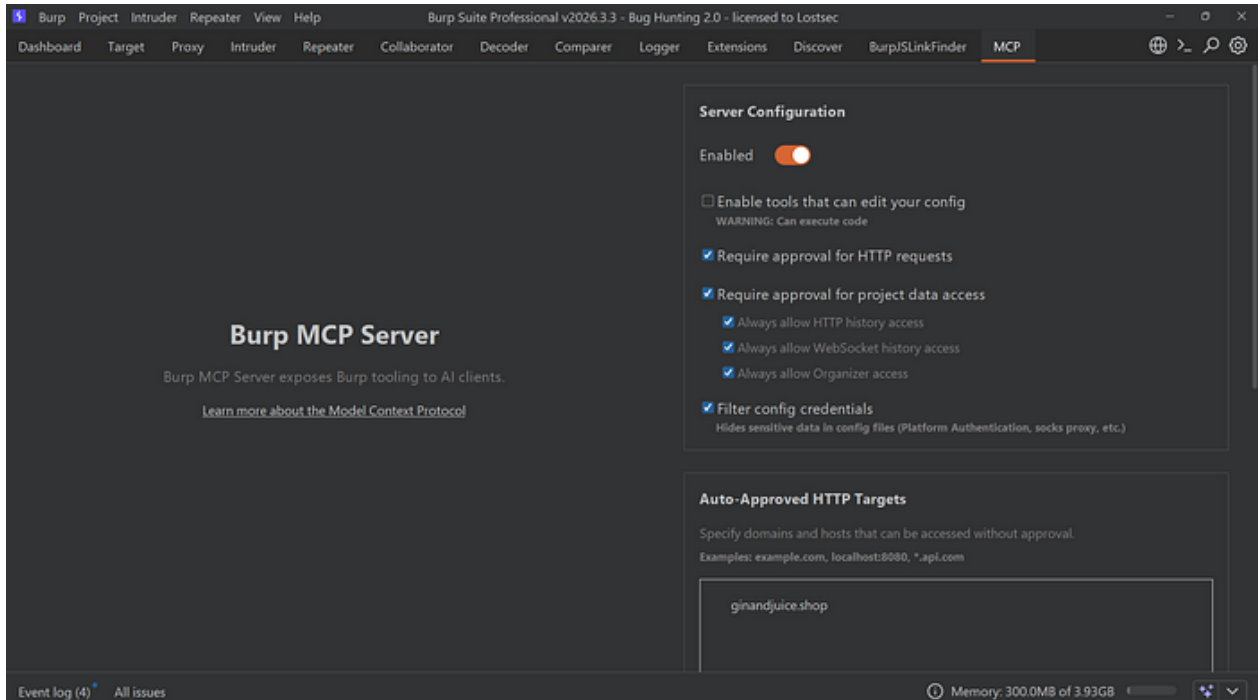
Step 1: Setting Up the Burp Suite MCP Server

Let's get everything set up. For this walkthrough, I'm using a dedicated test website so I can safely experiment with the setup without affecting any real-world targets.

1. **Install the Extension:** Open Burp Suite, navigate to the **BApp Store** and search for "MCP." Verify the extension's features and author, then click **Install**. It should open in a new tab once finished.

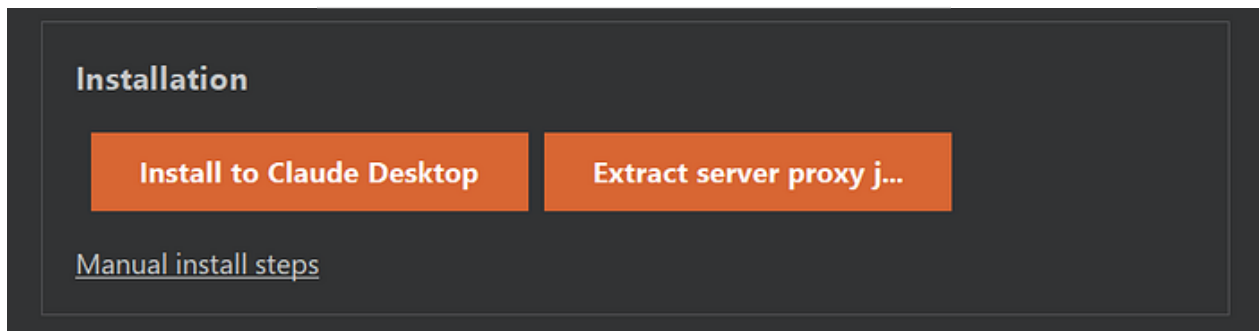


2. Configure Auto-Approved Targets: In the new MCP tab, enable the required options. You will see a setting for Auto-Approved HTTP Targets. Add your target domain, subdomains or use a wildcard (e.g., *.test-target.com). Note: This step is crucial. It ensures Claude won't pause to ask for permission every single time it interacts with that host.



3. Get the JAR File:

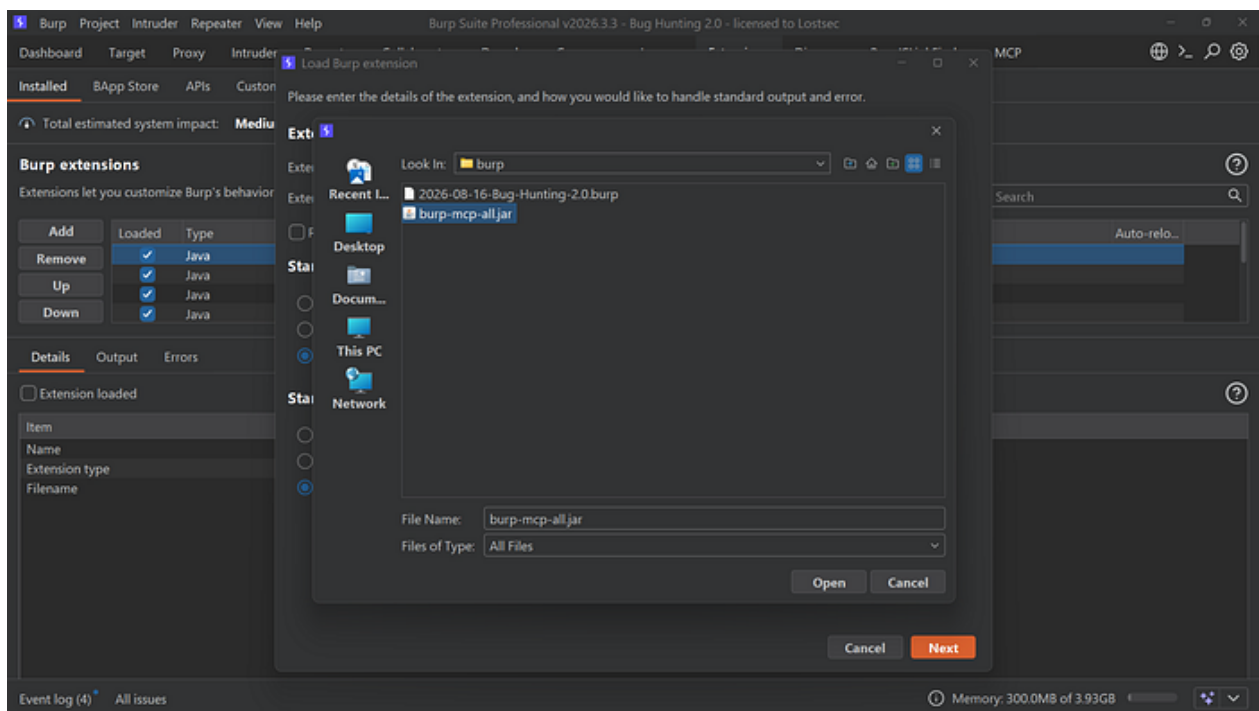
- If you are using Claude Desktop, you can usually follow the built-in installation button.



- If you are running Claude from your terminal, you need the compiled JAR file. You can extract this from the extension or generate it by following the build instructions on the project's official GitHub repository.

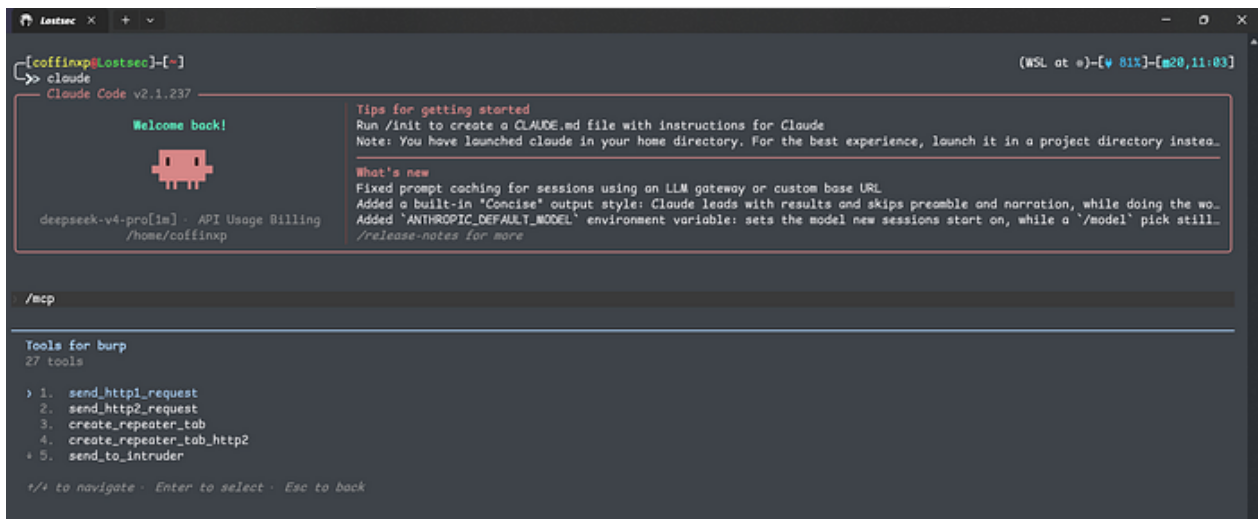
GitHub - PortSwigger/mcp-server: MCP Server for Burp (<https://github.com/PortSwigger/mcp-server>) MCP Server for Burp. Contribute to PortSwigger/mcp-server development by creating an account on GitHub.

1. **Load the Extension:** Go to the Extensions tab in Burp Suite and load the MCP JAR file. Once loaded, verify that the MCP server is running on localhost with its specific assigned port.



1. **Connect Claude:** Finally, open your terminal and enter the connection command provided by the extension to link Claude to your local Burp MCP server.

```
claude mcp add burp --transport sse http://127.0.0.1:9876
```

If you navigate to the MCP section inside Claude, you should now see your Burp MCP server listed. Clicking **View Tools** will reveal all the Burp functionalities Claude now has access to.

Step 2: Traffic Collection and Reconnaissance

Before we start testing with AI, we need to provide it with enough data to understand the target and its attack surface.

1. **Set the Scope:** Open your target website, ensure Burp's Proxy is enabled, and add your target to the scope in the **Target** tab.
2. **Filter the History:** Go to your HTTP history filter and select "**Show only in-scope items.**" This keeps the noise out and ensures Burp only tracks relevant traffic.
3. **Manual Crawling:** Browse the target application like a normal user. Click buttons, fill out forms, test search fields, log in, log out and interact with as many features as possible. The goal is to generate your HTTP history with a rich dataset of URLs, API endpoints, and parameters.
4. **Automated Discovery (Optional but Recommended):** To dig deeper, go to the Target tab, open **Engagement Tools** and select **Discover Content**. Let Burp crawl the app to find hidden directories and endpoints.
5. **Refine the View:** Once you have enough traffic, stop the crawl. Go back to your HTTP history filter and check "**Show only parameterized requests.**" You are now looking at the prime candidates for security testing.

Step 3: The AI Bug Hunting Workflow

Now comes the interesting part. We're going to use Claude Code to analyze the data we just gathered.

Phase 1: Analyzing and Organizing Traffic

Instead of scrolling through hundreds of raw requests, we use our first prompt to tell Claude:

Prompt 1 — List & understand the history

List the requests in my Burp proxy history, showing method, URL, and status code for each. Then do all of this in order:

- 1) Summarize what this application is and its main features.
- 2) Filter history to only ginandjuice.shop requests and highlight the most interesting ones (login, search, cart, checkout, APIs).
- 3) List all API endpoints in a table with method, path, and parameters.
- 4) Scan history for sensitive data – passwords, tokens, API keys, emails, personal info.
- 5) Give me a final summary of what this app does and which endpoints are worth testing.

Thought for 11s (ctrl+o to expand)

• Burp Proxy History Analysis - ginandjuice.shop

1) Request listing (260 total, deduplicated by method/URL/status)

ginandjuice.shop - 189 requests. Full unique listing:

Method	Status	Path	Notes
GET	200	/, /catalog, /blog, /about, /login, /my-account, /catalog/cart	Page views (some x25)
POST	200/302	/login	1 fail (no password → 200), 1 success → 302 /my-account
POST	302	/catalog/cart	Add-to-cart (x2)
POST	200	/catalog/product/stock	XML body (x2)
POST	302	/catalog/cart/coupon	x2 - one invalid, one accepted
POST	303	/catalog/cart/checkout	x2 - both → cart?err=FLAGGED
POST	200	/catalog/subscribe	JSON body
GET	200	/catalog/product?productId=1	x3
GET	200	/catalog?searchTerm=test	Search
GET	200	/order/details?orderId=0254809	Full PII in response

Claude will analyze the data, remove duplicates, and give you a clean overview of the application. It will identify API endpoints, interesting parameters, and any sensitive information exposed in the traffic.

Phase 2: Mapping the Attack Surface

Next, we'll ask Claude to look for potential areas where vulnerabilities could be hiding:

Prompt 2 — Pick the attack surface

Analyze my Burp proxy history for `ginandjuice.shop` and identify the attack surface. In order:

- 1) Find all requests with search or q parameters – injection candidates.
- 2) Find all requests with numeric ID parameters (like `id=1`, `/api/xxx/1`, order numbers) – IDOR candidates.
- 3) Find `admin`, `profile`, `user`, `account`, `upload`, and API endpoints.
- 4) Find any JavaScript files in history and fetch the interesting ones, looking for hardcoded secrets, API paths, or clues.
- 5) Output a clean prioritized list: the top 6 endpoints to test for bugs, grouped by bug type (SQLi, XSS, IDOR, auth).

```

• Attack Surface Analysis - ginandjuice.shop

Coverage: parsed 100% of the fresh Burp history dump (417 lines → 209 entries, 0 parse failures) via scripted extraction; 189 entries are ginandjuice.shop. Inventory is unchanged from last session - no new requests.

1) Search / q parameters - injection candidates

```

Request	Evidence
GET /catalog?searchTerm=test → 200	Only search-driven request in history. Backed by searchLogger.js, which POSTs the params to a hidden /logger endpoint (see §4)

```

No q-style params seen on other endpoints, but searchTerm is the canonical PortSwigger SQLi vector here.

2) Numeric ID parameters - IDOR candidates

```

Request	Status	Notes
GET /catalog/product?productId=1	200	×3 - classic SQLi/IDOR param
GET /order/details?orderId=8254889	200	Returns full PII: Carlos Montoya, 17 Rosewood Crescent, Hampstead, London AB12 3CD + card **** 5613

```

orderId is the hot one - sequential numeric order numbers exposing other users' billing/shipping data.

```

Claude turns the raw URL list into a clear attack map, showing which requests are worth investigating for different vulnerability types.

Phase 3: Active Vulnerability Testing

This is where we move into the **actual security testing**. We'll ask Claude to test the relevant endpoints and vulnerability types identified in the previous phase.

Prompt 3 — Test for bugs (the main show)

Using the endpoints from the attack surface list, test `ginandjuice.shop` for vulnerabilities, one bug class at a time. Do them in this order and report a one-line result after each test:

- 1) SQL injection – test the search endpoint with a single quote, OR and UNION SELECT payloads.
- 2) Reflected XSS – inject `<script>alert(1)</script>` into search and product parameters, check if it's reflected unencoded.
- 3) IDOR – take order/profile requests and swap the ID for neighboring numbers (1, 2, 3); compare response lengths to confirm access to other users' data.
- 4) Weak/default credentials – test login with common combos like `admin/admin`, `admin@ginandjuice.shop:admin123`, and note the exact error messages (user enumeration).
- 5) User enumeration – compare login responses for a fake email vs an existing email.
- 6) Rate limiting – send the login request 10 times fast; is there any lockout or rate limit?
- 7) Broken access control – try hitting admin-only API endpoints (like `/api/Users` or `/api/Products`) unauthenticated.
- 8) Business logic – try common coupon codes like GIN or JUICE and check if the discount can be reused.

At the end, list every confirmed or suspected finding with severity, the endpoint, and which request proved it.

This phase can take some time. While Claude is running, press **Ctrl+O** to view its background processes. You'll be able to watch Claude send requests through Burp Suite using the MCP connection, test different parameters, and analyze the resulting HTTP responses.

One-line results per test			
#	Test	Result	
1	SQLi (search)	CONFIRMED - category=gin' OR 1=1-- returns all 18 products vs 9 for category=gin; searchTerm=X throws HTTP 500 (raw SQL)	
2	Reflected XSS	Not confirmed - <script> is HTML-encoded on reflection; quote-breakout dies on the SQL 500 first; productId rejects non-numeric input	
3	IDOR	Neighbors 0254807-0254811 all "Order not found" - but orderId=0254809 with zero cookies returns 200 + full PII (missing authz)	
4	Weak/default creds	No default login - admin/admin = 500 anomaly; real password is disclosed by the app itself (see findings)	
5	User enumeration	Not confirmed - generic "Invalid username or password." for fake and real users alike	
6	Rate limiting	ABSENT - 5 rapid wrong-password attempts all processed, no lockout, no 429, no CAPTCHA	
7	Access control	/admin + 403 "Access denied" (exists, protected); /api/* + 404; /vulnerabilities + 200 public page disclosing account credentials + full vuln map	
8	Coupons	GIN/JUICE invalid; 9in&JUICE5H0P valid (30%: \$30.50 + \$24.40); reuse blocked ("Coupon already applied" - no stacking via direct replay)	

Confirmed / suspected findings			
Severity	Finding	Endpoint	Proving request
High - Confirmed	SQL injection in WHERE clause	GET /catalog?category=	category=gin' OR 1=1-- → 170838, all 18 products (vs 131028 / 9 products for gin)
High - Confirmed	Missing authentication on order details (PII disclosure)	GET /order/details?orderId=	Request with no cookies, orderId=0254809 → 200 with name, full address, card **** 5613
High - Confirmed	Credential disclosure in login flow	POST /login (step 2)	username=carlos + page renders Password hunter2; also /vulnerabilities page publishes the same creds
Medium - Confirmed	No rate limiting / brute-force protection on login	POST /login	5x wrong passwords in ~90s + all 200 with generic error, no lockout

Once the testing is complete, Claude will provide a list of the vulnerabilities it tested, separating confirmed findings from suspected issues. It will also include the relevant HTTP requests and supporting evidence for each finding.

Phase 4: Generating the Final Report

Finally, we turn the validated findings into a **clear, professional security report**, including the affected endpoints, technical evidence, impact, severity, and recommended remediation.

Prompt 4 — Wrap up like a pro

Write a professional pentest-style summary report of everything we found on ginandjuice.shop. For each finding include: vulnerability name, severity (Critical/High/Medium/Low), affected endpoint, how it was exploited, and the proof (request + response). End with a prioritized fix list, most critical first. Also save the interesting requests to Burp Organizer for evidence.

```

Target: https://ginandjuice.shop (Gin & Juice Shop, PortSwigger scanner-evaluation lab) · Date: 2026-08-16 · Tooling: Burp Suite Professional (MCP-driven HTTP/1.1 requests, all logged in Proxy history + Repeater tabs)
Scope: Full application surface discovered via proxy history: catalog, search, products, cart/coupons/checkout, login/account, orders, blog, subscription, admin/API paths, 35 assets.

---

Findings

F-01 · SQL Injection in category Parameter - High (Confirmed)

- Endpoint: GET /catalog?category=
- Exploitation: The category filter is concatenated into a SQL WHERE clause without parameterization. Injecting Gin' OR 1=1-- broke out of the string literal and neutralized the filter, returning the entire product table.
- Proof:
  - Baseline: GET /catalog?category=Gin HTTP/1.1 + 200, Content-Length 13102, 9 product tiles (Gin category only)
  - Attack: GET /catalog?category=Gin%27%20OR%201%3D1-- HTTP/1.1 + 200, Content-Length 17083, all 18 products (productId 1-18) rendered:
    Input hidden type text name "category" value "Gin&apos; OR 1=1--"
    section class "container-list-tiles" ... 18 a href "/catalog/product?productId=N" tiles ...
- Impact: Unauthorized data retrieval (union-based exfiltration of orders/users likely with further enumeration).

F-02 · SQL Error Handling / Injection Surface in searchTerm - Medium (Confirmed surface, unexploited further)

- Endpoint: GET /catalog?searchTerm=
- Exploitation: The search term is also used raw in SQL. A bare X (unescaped LIKE wildcard) and 'alert(1);// both crashed the query engine; a lone ' altered the result set (returned the 9-product Gin set).
- Proof:
  - GET /catalog?searchTerm=X HTTP/1.1 + 500, Content-Length: 3761, page body Internal Server Error
  - GET /catalog?searchTerm=X%27%3Balert(1)%3B%2F HTTP/1.1 + 500, same error page
  - GET /catalog?searchTerm=X%27 HTTP/1.1 + 200, 130838, product grid returned
- Impact: Confirms unparameterized SQL; combined with F-01 raises confidence of full injection (blinds/UNION) on both parameters.

F-03 · Missing Authentication on Order Details (PII Disclosure) - High (Confirmed)

- Endpoint: GET /order/details?orderId=
- Exploitation: The endpoint performs no session check whatsoever. A request with no cookies at all returns full order PII for a known order number.
- Proof:
  GET /order/details?orderId=0254009 HTTP/1.1
  Host: ginandjuice.shop
  (no Cookie header)
  + 200, Content-Length: 2747;
  
```

Within seconds, Claude generates a clean, well-organized report that's ready for a bug bounty submission.

The One-Shot Prompt

If running all four phases separately feels too time-consuming, you can combine them into a single **"One-Shot" prompt**. This tells Claude to handle the entire workflow in one go, from analyzing the data and mapping the attack surface to testing for vulnerabilities, validating the findings and generating the final report.

ROLE

You are a senior bug bounty hunter running a structured engagement against a live demo e-commerce **target**. You work methodically, you validate everything **with real** PoCs, **and** you **never** inflate severity. Every phase below builds **on** the previous one – do **not skip** ahead.

SCOPE & RULES

- **Target**: <https://ginandjuice.shop/> only. **Ignore** every other host **in** Burp history.
- **Primary** toolkit: Burp MCP (proxy history, regex filtering, Repeater, Collaborator, **encoding**). Direct **HTTP** calls only **as** a fallback.
- **Time-box** each attempt: two clean exploitation tries per bug class, **then move on**. Dead ends are normal – **log** the negative result **and continue**.
- **Evidence-first**: **no** finding enters the report **without** a reproducible request/response pair.

PHASE 1 – PRE-ENGAGEMENT INTEL & SURFACE MODELING

1. Build the operational picture **from** proxy history: inventory every host, **page**, **endpoint**, **and** **API route**; note methods **and** parameters.
2. Fingerprint the stack: **server** headers, framework signatures, cookies (JWT? **session** cookie flags?), JS libraries, CDN/WAF presence.
3. Mine the client side: **fetch** the JavaScript bundles **in** history **and** extract API paths, hardcoded secrets, **and** app logic clues (this **is where** hidden endpoints live).
4. Map the auth model: **login** flow, token lifecycle, **password reset** behavior, registration, **role** structure, **and** any **session**-related headers.
5. Regex-scan history **for** high-value signals: **id=**, **search/q**, **admin|user|profile|upload** paths, token/**secret** patterns, **and** sensitive **data** leakage.
6. Produce an attack surface model: a scored list of endpoints **with** the bug classes most likely **to** succeed **on** each. **No** findings yet – this **is** intel only.

PHASE 2 – ATTACK HYPOTHESES & PRIORITIZATION

- Rank the attack surface **by** (impact × likelihood) ÷ effort. E-commerce priority: business logic **and** access **control before** generic injection.
- Write **5-8** concrete attack hypotheses, **e.g.**: "the coupon endpoint accepts replay of a consumed code", "order IDs are sequential and unauthenticated reads leak customer data", "the search endpoint reflects input into the page without encoding".
- **State** your expected evidence **for** each hypothesis **before** testing – a clean test needs a falsifiable claim.

PHASE 3 – EXPLOITATION (in this order)

1. Business logic: cart/price manipulation (negative quantities, tampered price fields, currency swaps), coupon logic (reuse, **self-referral**, rate), checkout **state** manipulation (skipping steps, reordering the flow).
2. IDOR / broken **object-level** access: enumerate neighboring IDs **on** orders, profiles, **and** API objects; verify **with** response-content comparison, **not** just **status** codes.
3. **Authentication & session**: **default/weak** creds, user enumeration, missing rate limiting, **session** fixation/invalidation gaps, token weaknesses.
4. Access **control**: unauthenticated calls **to** user/**admin** APIs, privilege escalation via **role** or flag tampering, **forced** browsing.
5. Injection: SQLi (classic, **UNION**, **and** blind via timing/**boolean**) **on** search **and** parameters; XSS (reflected **in** search, stored **in** reviews/**profile** fields, DOM sinks **in** JS); SSTI **and** command injection **on** file/export/convert features.
6. **Out-of-band**: inject Collaborator payloads **into** any **URL/fetch/image/import** parameters

for blind SSRF and XXE; poll for DNS/HTTP interactions.

7. File upload & misc: extension/filter bypasses, content-type spoofing, same-origin serving, plus info disclosure (source maps, debug endpoints, backup files, verbose errors).

PHASE 4 – CHAINING & IMPACT AMPLIFICATION (the part that makes findings severe)

- For every confirmed primitive, map what it unlocks: can an IDOR pivot into account takeover? Can stored XSS capture an admin session? Can the coupon bug chain with registration into unlimited free orders? Can a leaked API key from JS escalate to admin API access?

- Follow each chain to its maximum impact and document the full kill chain – a chain of two Mediums often reports as High/Critical.

- Sweep for variant coverage: same bug class on other endpoints/parameters – one pattern, many surfaces.

PHASE 5 – VALIDATION GATE (before anything enters the report)

- Replay every finding with a clean, minimal request in Repeater; it must reproduce deterministically, twice.

- Prove impact in the response: leaked data, executed code, state change, privileged action – no theoretical impact statements.

- Kill any finding you can't stand behind. A report with 3 confirmed issues beats 12 speculative ones.

- Store all PoC requests in Burp Organizer for the evidence appendix.

PHASE 6 – PROFESSIONAL REPORT (triager-ready)

Structure the final output exactly like a production submission:

1. Executive summary – 3-4 lines: what the app is, what you found, overall risk posture.
2. Findings – one entry per confirmed issue: Title / Severity (CVSS 3.1 vector + score) / Affected endpoint / Vulnerability class / Steps to reproduce (numbered, copy-pasteable) / Evidence (request + response excerpts) / Business impact (what an attacker actually gains) / Remediation (specific fix, not generic advice).
3. Chained findings – for each chain, the component bugs and the escalation path.
4. Summary table – all findings ranked by severity with a one-line fix each.
5. Appendix – the Organizer request IDs as raw evidence.

If the engagement produced no confirmed vulnerabilities, say exactly that, and close with the top 3 residual risks you observed with recommended hardening – a clean report is a credible report.

• Phase 6 complete. The full report is saved at `/home/coffinxp/engagement/report.md`. Final deliverable:

Security Assessment Report – Gin & Juice Shop (ginandjuice.shop)

Engagement date: 2026-08-16 · Scope: `https://ginandjuice.shop/` only · Tooling: Burp Suite (history, Repeater, Collaborator), headless Chromium validation, HTTP replay

1. Executive Summary

Gin & Juice Shop is a deliberately-vulnerable e-commerce demo application (PortSwigger scanner benchmark) running a Java backend on nginx behind AWS ALB. The engagement confirmed **three vulnerabilities**: unauthenticated access to order details exposing customer PII and payment-card data (**High**), an unlimited-replay coupon granting a permanent 20% discount to anyone (**Medium**), and default credentials on the seeded customer account (**Medium**). A full sweep of injection (SQLi/XOR/XSS/SSRF), access-control, and business-logic surfaces produced no additional exploitable findings; several hardening gaps (no CSP/HSTS, no login rate limiting) are reported as residual risks.

2. Findings

F1 – Unauthenticated Access to Order Details Exposes Customer PII and Card Data (High)

Field	Value
Severity	High – CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N = 7.5
Affected endpoint	GET <code>/order/details?orderId=<id></code>
Vulnerability class	Broken Object Level Authorization / Missing Access Control (OWASP A01)

Steps to reproduce:

- Open `https://ginandjuice.shop/order/details?orderId=0254800` in an anonymous session (no login, no cookies).
- HTTP 200 with the full order record: name, billing/shipping address, masked card, totals, date.
- Repeat with 0254685, 0254725, 0254774, 0254791 – all render unauthenticated.
- Negative control: unknown ID (0254808) → 400 – endpoint is live and data-dependent.

Evidence:

+ Gathering intel and modeling attack surface... (5m 4s · ↓ 18.0k tokens)

- └ ☒ Phase 1: Pre-engagement intel & surface modeling
- ☐ Phase 2: Attack hypotheses & prioritization
- ☐ Phase 3: Exploitation
- ☐ Phase 4: Chaining & impact amplification
- ☐ Phase 5: Validation gate
- ... +1 pending

3. Chained Findings

Chain	Components	Escalation path	Net severity
A – Customer data breach	F1	No auth or interaction; any known/guessed order ID → full PII + payment metadata (IDs leak via receipts/account links)	High (standalone)
B – Discount factory	F3 + subscribe → F2	carlos:hunter2 → mint static coupon → 20% off every cart forever. Checkout completion currently blocked by account-level <code>err=FLAGGED</code> ; if the flag is account-scoped, any fresh account completes the chain – unlimited discounted orders	Medium, rises to High if flag is bypassable/absent
C – Log injection	/logger (hidden, unauth)	Attacker-controlled JSON accepted unvalidated; sink not observable from outside	Informational

Because the one-shot prompt runs the entire workflow in a single process, it may take some time, especially when there's a large amount of HTTP history to analyze. Let Claude work through each phase, and once it's finished, you'll have a complete security report with validated findings and potential vulnerability chains.

Bonus: Bug Hunting Without Burp MCP

In my previous video, I showed how Claude Code can work directly with individual security artifacts, such as Burp raw request files and JavaScript **files**. This approach is much simpler when you only want to investigate a specific request, source file, or piece of reconnaissance data.

So, if you don't want to set up **Burp MCP**, you can still build a powerful workflow by giving Claude Code the exact data you want it to analyze.

Here are some of the most useful shortcuts:

1. Analyze a Burp Raw Request

Save any interesting Burp request as a .txt file, then give it to Claude and let it analyze the request for you.

Here's a realistic example of a **Burp Suite raw request** you can use for the demo.

```
POST /api/v1/auth/signup HTTP/2
Host: demo.local
Cookie: session=eyJhbGciOiJIUzI1NiJ9.demo.session
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 Chrome/139.0.0.0 Safari/537.36
Accept: application/json, text/plain, */*
Accept-Language: en-US,en;q=0.9
Content-Type: application/json
Origin: https://demo.local
Referer: https://demo.local/register
X-CSRF-Token: demo-csrf-token
X-Forwarded-For: 127.0.0.1
Connection: keep-alive

{
  "username": "john.doe",
  "email": "john@example.com",
  "password": "Password123!",
  "confirmPassword": "Password123!",
  "role": "user",
  "inviteCode": "",
  "referrer": "newsletter"
}
Identify:
1. All user-controlled inputs and parameters.
2. Authentication and authorization mechanisms.
3. Interesting IDs and object references.
4. Potential IDOR/BOLA candidates.
5. Injection points.
6. Business-logic parameters.
7. Security-relevant headers and cookies.
8. Any unusual behavior that deserves further testing.

Rank each issue by severity. For each interesting point, explain why it is worth investigating and suggest manual testing steps.
Do not assume a vulnerability exists.
Separate observations from hypotheses.
```

This is useful when you have a particularly interesting request and only want Claude to focus on that specific request instead of your entire Burp history.

2. Analyze a JavaScript File

JavaScript analysis is another useful shortcut. Instead of connecting Claude to Burp, you can simply give it a JavaScript file and let it analyze the code directly.

Analyze this JavaScript file from a bug bounty perspective.

Extract and categorize:

1. API endpoints and URL paths.
2. HTTP methods.
3. Parameters and parameter names.
4. Authentication and authorization logic.
5. Hardcoded secrets, tokens, or API keys.
6. Administrative or internal endpoints.
7. Upload and download functionality.
8. Debug or development endpoints.
9. Hidden functionality and feature flags.
10. Interesting DOM sources and sinks.
11. Third-party integrations.

For every interesting discovery, explain why it deserves further investigation.

Do not report something as a vulnerability unless there is enough evidence to support it.

3. Testing a JavaScript File Directly from a URL

You don't always need to download a JavaScript file manually. If you find a .js file during recon, you can give the URL directly to Claude and ask it to analyze the file.

This is useful when you find an interesting JavaScript bundle and want to quickly identify API endpoints, hidden routes, sensitive information and other security-relevant clues.

Use a prompt like this:

Analyze this JavaScript file for security-relevant information:

<JS_URL>

Look for:

1. API endpoints and hidden routes
2. Interesting parameters
3. Authentication or authorization logic
4. Hardcoded secrets or sensitive information
5. Internal or administrative functionality
6. Debug or development endpoints
7. Client-side security issues
8. Anything else that could be useful for authorized security testing

For each finding, explain why it is interesting and provide the relevant code or URL as evidence.

4. Analyze Multiple JS Files

If you've already collected JavaScript files during recon, put them into a same directory and let Claude analyze them together.

Analyze every JavaScript file in this directory.
Build a consolidated attack-surface map containing:

1. API endpoints
2. HTTP methods
3. Parameters
4. Authentication functionality
5. Authorization logic
6. Administrative functionality
7. Upload/download features
8. WebSocket endpoints
9. Third-party integrations
10. Hardcoded secrets or credentials
11. Interesting client-side sinks
12. Hidden or undocumented functionality

Deduplicate identical endpoints and group related functionality together.

For each discovery, include relevant details and explain why it is interesting from a security-testing perspective.

At the end, rank the top 10 discoveries that deserve manual security testing and explain why each one is interesting.

Do not assume a vulnerability exists. Clearly separate observed functionality from security hypotheses.

This is especially useful for large applications where searching through many JavaScript files can take a lot of time.

5. Ask Claude to Investigate a Specific Vulnerability

You can also use Claude as a second opinion when you already have a suspicious request.

For example:

Analyze this request for a possible IDOR/BOLA vulnerability.

Explain:

1. Why this request could be interesting.
2. Which parameter controls the object being accessed.
3. What needs to change to test the authorization boundary.
4. What response differences would confirm the issue.
5. How I should manually validate the finding.

Do not assume the vulnerability exists.

Clearly distinguish observations from hypotheses.

Explain what evidence would be required to confirm the issue.

This works well when you've already discovered something manually and want Claude to help structure the investigation.

6. Review Source Code

If the source code is available, Claude Code can go beyond recon and help perform a deeper security review of the code itself.

Review this source code from a security perspective.

Identify:

1. Authentication weaknesses.
2. Authorization and access-control issues.
3. Input validation problems.
4. Injection risks.
5. Sensitive information exposure.
6. Insecure file handling.
7. Session-management issues.
8. Business-logic flaws.

For every confirmed or evidence-supported issue:

- Explain the root cause.
- Point to the relevant code.
- Explain the security impact.
- Suggest a secure fix.
- Explain how the issue could be manually validated in an authorized environment.

Do not report theoretical issues without explaining the evidence.

Clearly distinguish confirmed issues, security-relevant observations, and hypotheses.

Prioritize findings by severity and practical exploitability.

This gives you another workflow that doesn't require Burp at all.

7. Turn Everything Into a Bug Bounty Report

Once you've finished your investigation, you can give Claude the evidence and let it organize everything into a clear report.

Based on the provided HTTP requests, JavaScript analysis, recon results, and testing notes, create a professional bug bounty report.

For each confirmed finding, include:

- Title
- Severity
- Affected endpoint
- Vulnerability class
- Steps to reproduce
- Evidence
- Security impact
- Remediation

Structure the report clearly and professionally.

Separate:

1. Confirmed vulnerabilities
2. Security-relevant observations
3. Potential issues requiring further validation

Do not include speculative findings as confirmed vulnerabilities.

Only classify an issue as a confirmed vulnerability when the provided evidence demonstrates an actual security impact.

Clearly distinguish observed behavior from security conclusions.

Rank confirmed findings by severity and practical impact.

The Full Shortcut

The real advantage comes when you combine these approaches.

You can give Claude Code:

```
project/
├─ recon/
│   ├── subdomains.txt
│   ├── alive.txt
│   └─ urls.txt
├─ requests/
│   ├── login.txt
│   ├── profile.txt
│   └─ api.txt
├─ javascript/
│   ├── app.js
│   ├── dashboard.js
│   └─ auth.js
└─ source/
    └─ application-code/
```

Then ask Claude to connect the dots and correlate everything:

You are assisting with an authorized security assessment.
Analyze the files in this project and correlate the information across recon, HTTP requests, JavaScript, and source code.

Build:

1. An attack-surface map.
2. An API inventory.
3. An authentication and authorization map.
4. A list of interesting parameters and object IDs.
5. Hidden functionality discovered in JavaScript or source code.
6. Potential vulnerability hypotheses.
7. A prioritized manual testing plan.
8. Confirmed findings supported by the provided evidence.
9. A professional security report.

Do not treat suspicious patterns as confirmed vulnerabilities.

For every finding, clearly explain the evidence that supports it and what additional validation is required.

This gives you a lightweight alternative to the MCP workflow.

MCP is useful when you want Claude to interact directly with Burp and work continuously with live traffic. But if you only want Claude to analyze specific artifacts, these shortcuts can be much faster and easier to set up.

You can also watch this video where I showed the complete practice of this method:

[Embedded content](https://cdn.embedly.com/widgets/media.html?src=https%3A%2F%2Fwww.youtube.com%2Fembed%2FVr-NEWVLN9Y%3Ffeature%3Doembed&display_name=YouTube&url=https%3A%2F%2Fwww.youtube.com%2Fwatch%3Fv%3DVr-NEWVLN9Y&image=https%3A%2F%2Fi.ytimg.com%2Fvi%2FVr-NEWVLN9Y%2Fhqdefault.jpg&type=text%2Fhtml&schema=youtube) (https://cdn.embedly.com/widgets/media.html?

src=https%3A%2F%2Fwww.youtube.com%2Fembed%2FVr-

NEWVLN9Y%3Ffeature%3Doembed&display_name=YouTube&url=https%3A%2F%2Fwww.youtu

be.com%2Fwatch%3Fv%3DVr-NEWVLN9Y&image=https%3A%2F%2Fi.ytimg.com%2Fvi%2FVr-

NEWVLN9Y%2Fhqdefault.jpg&type=text%2Fhtml&schema=youtube)

Claude Code Skills for Bug Hunting

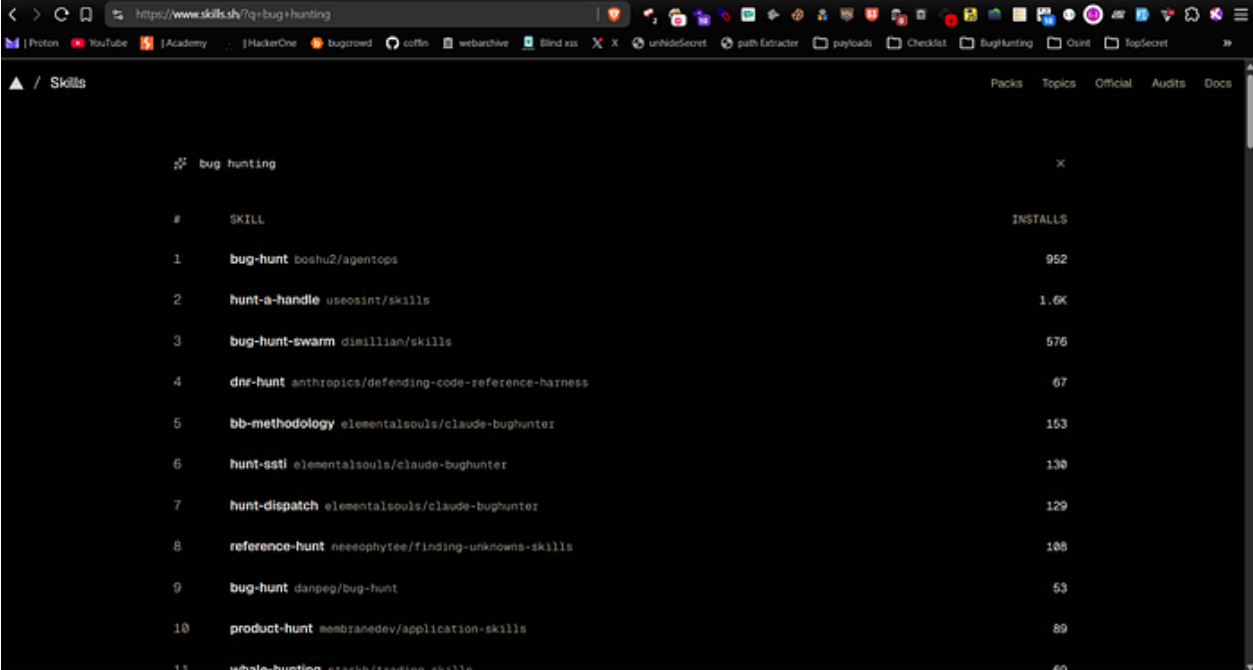
Claude Code Skills let you package a bug-hunting methodology into a reusable workflow. Instead of giving Claude the same prompts and instructions repeatedly, you can define the methodology, tools, workflow, payloads and references inside a Skill and reuse it whenever you need it.

Think of a Skill like a Nuclei template, but for Claude. Instead of defining a vulnerability check, a Skill defines the methodology Claude should follow for a specific task.

Skills are especially useful for repetitive bug-hunting workflows because you can build the process once and then reuse it across different authorized targets.

Using Open-Source Bug Hunting Skills

You don't always need to create every Skill yourself. There are already open-source Skills created by the community that you can install and use with Claude Code.



The screenshot shows the Skills.sh website with a list of skills. The table below represents the data visible in the image.

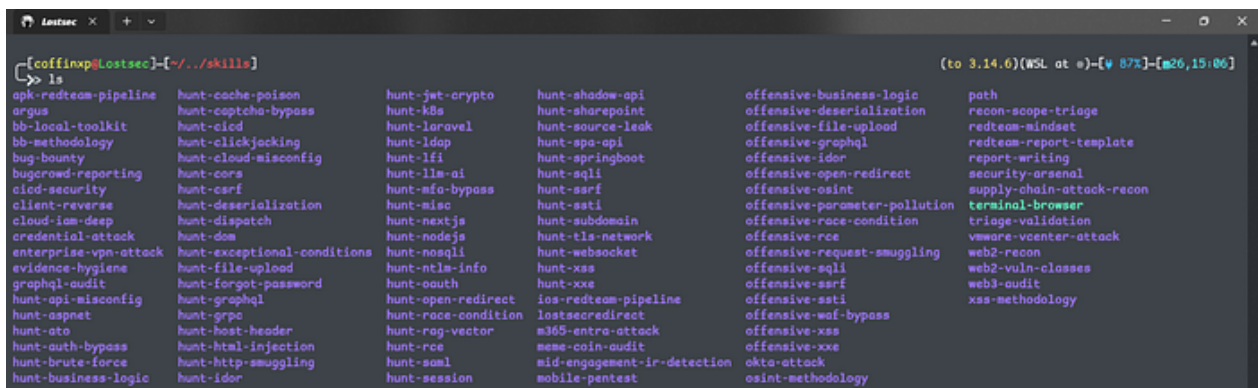
#	SKILL	INSTALLS
1	bug-hunt boshu2/agentops	952
2	hunt-a-handle useosint/skills	1.6K
3	bug-hunt-swarm dimillian/skills	576
4	dnr-hunt anthropics/defending-code-reference-harness	67
5	bb-methodology elementalsouls/claude-bughunter	153
6	hunt-ssti elementalsouls/claude-bughunter	130
7	hunt-dispatch elementalsouls/claude-bughunter	129
8	reference-hunt neeeophytee/finding-unknowns-skills	108
9	bug-hunt daneg/bug-hunt	53
10	product-hunt membranede/application-skills	89
11	whale-hunting atash/trading-skills	69

The Agent Skills Directory (<https://www.skills.sh/>) Discover and install skills for AI agents.

You can visit **skill.sh** to browse available Skills and search for the type of workflow you need. You'll find different collections covering areas such as OSINT, bug hunting and offensive security.

Adding GitHub-Hosted Bug Hunting Skills

Apart from skill.sh, you can also find useful bug-hunting Skills directly on GitHub. Many security researchers publish their own Skills and complete collections that you can add to Claude Code.



```
[coffinx@lostsec] ~/skills (to 3.14.6) [WSL at e] [x 87%] [m26,15:06]
>> ls
apk-redteam-pipeline  hunt-cache-poison      hunt-jwt-crypto        hunt-shadow-api        offensive-business-logic  path
argus                 hunt-captcha-bypass    hunt-k8s               hunt-shorepoint        offensive-deserialization  recon-scope-triage
bb-local-toolkit      hunt-cicd              hunt-laravel           hunt-source-leak       offensive-file-upload     redteam-mindset
bb-methodology        hunt-clickjacking      hunt-idap              hunt-spo-api           offensive-graphql         redteam-report-template
bug-bounty            hunt-cloud-misconfig   hunt-ifi               hunt-springboot        offensive-idor            report-writing
bugcrowd-reporting    hunt-cors              hunt-llm-ai            hunt-aql               offensive-open-redirect   security-arsenal
cicd-security         hunt-csrf              hunt-mfa-bypass        hunt-asrf              offensive-osint           supply-chain-attack-recon
client-reverse        hunt-deserialization   hunt-misc              hunt-astl              offensive-parameter-pollution  terminal-browser
cloud-iam-deep        hunt-dispatch          hunt-nextjs            hunt-subdomain         offensive-race-condition    triage-validation
credential-attack     hunt-dom               hunt-nosqli            hunt-tls-network       offensive-rce              vmware-vcenter-attack
enterprise-vpn-attack hunt-exceptional-conditions  hunt-nodejs           hunt-websocket         offensive-request-smuggling  web2-recon
evidence-hygiene      hunt-file-upload       hunt-ntlm-info         hunt-xxe               offensive-sqli             web2-vuln-classes
graphql-audit          hunt-forgot-password   hunt-oauth             hunt-xxe               offensive-ssrf             web3-audit
hunt-api-misconfig     hunt-graphql           hunt-open-redirect     ios-redteam-pipeline   offensive-astl            xss-methodology
hunt-asnmap           hunt-grpc              hunt-race-condition    lostsecdirect          offensive-waf-bypass
hunt-ato              hunt-host-header       hunt-rag-vector        m365-entra-attack     offensive-xxe
hunt-auth-bypass      hunt-htnl-injection    hunt-rce               meme-coin-audit        offensive-xxe
hunt-brute-force       hunt-http-smuggling    hunt-saml              mid-engagement-ir-detection  okta-attack
hunt-business-logic    hunt-idor              hunt-session           mobile-pentest         osint-methodology
```

Claude-Red/Skills/web at main · SnailSploit/Claude-Red (<https://github.com/SnailSploit/Claude-Red/tree/main/Skills/web>) claude-red is a curated library of offensive security skills designed for the Claude skills system. Each skill is a...

GitHub - elementalsouls/Claude-BugHunter: A Claude Code skill bundle for bug hunting and external... (<https://github.com/elementalsouls/Claude-BugHunter>) A Claude Code skill bundle for bug hunting and external red-team work - 82 skills, 15 slash commands, 681...

GitHub - Awarexone/Agentic-Bug-Hunter: AI-powered bug bounty hunting toolkit that works with or... (<https://github.com/Awarexone/Agentic-Bug-Hunter>) AI-powered bug bounty hunting toolkit that works with or without subscription. - Awarexone/Agentic-Bug-Hunter

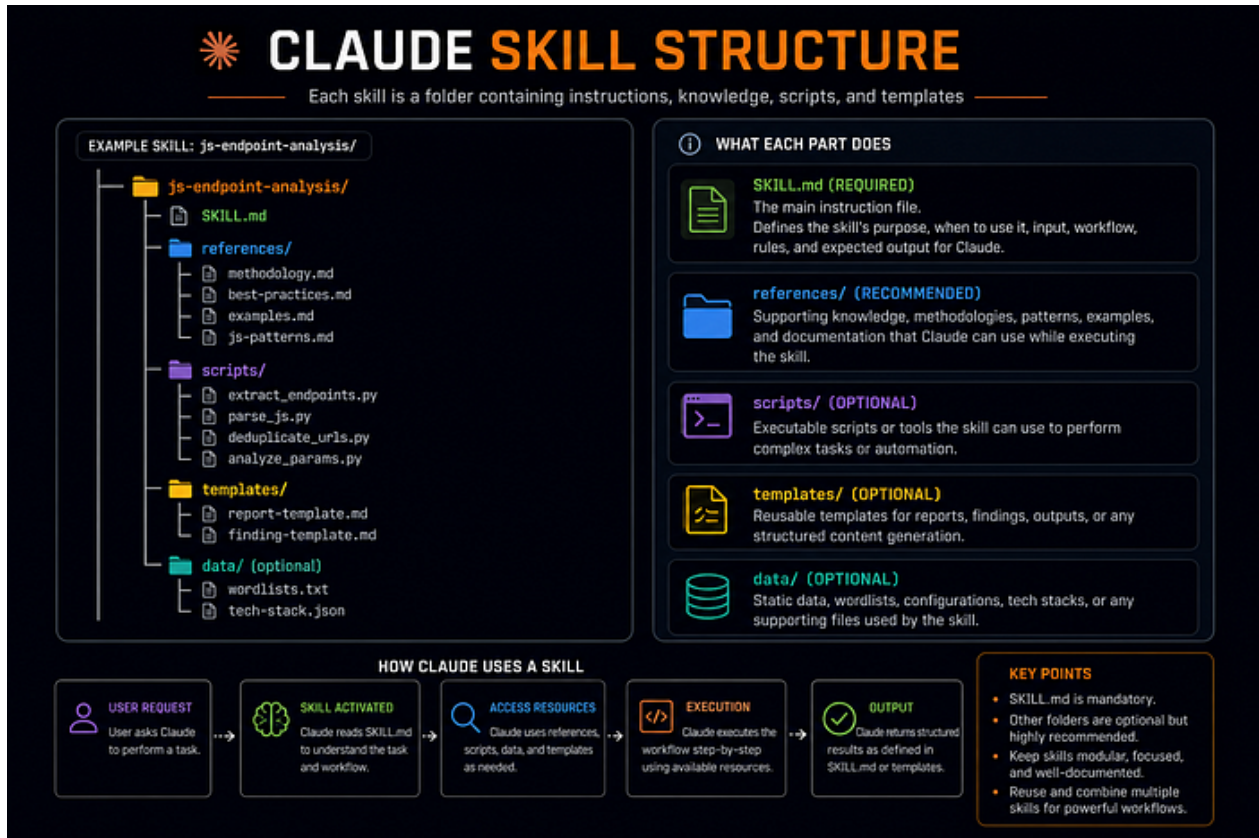
These repositories can contain Skills for specific tasks such as reconnaissance, vulnerability discovery, endpoint analysis, authentication testing, API testing and other parts of the bug-bounty workflow.

Creating Your Own Custom Bug Hunting Skills

The more interesting option is creating your own Skills from the methodologies you already use.

You can take an existing bug-hunting methodology, such as a research article, your own notes, a testing workflow or a collection of techniques, and convert it into a reusable Skill.

Claude Skill Structure & Format



```

1 |
2 name: bb-methodology
3 description: Use at the START of any bug bounty hunting session, when switching targets, or when feeling lost about what to do next. Master orchestrator that combines the 5-phase non-linear hunting workflow with the critical thinking framework (Developer psychology, anomaly detection, What-If experiments). Routes to all other skills based on current hunting phase. Also use when asking "what should I do next" or "where am I in the process."
4 |
5 |
6 # Bug Bounty Methodology: Workflow + Mindset
7 |
8 Master orchestrator for hunting sessions. Combines the 5-phase non-linear workflow with the critical thinking framework that separates top 1% hunters from the rest.
9 |
10 |
11 |
12 ## PART 0: MODE CONFIGURATION (Before Anything Else)
13 |
14 ==confirm the engagement type before deciding what counts as a finding== The same target produces a different report shape depending on which mode applies. Getting this wrong is the single biggest waste of time in this workflow - answer it explicitly before Phase 0.
15 |
16 | Engagement type | What counts as a finding | What gets rejected |
17 | --- | --- | --- |
18 | ==bug_bounty== (WS / Bugcrowd / Intigriti / private VDP) | Impact-demonstrated bugs ONLY. Full chain to attacker-attainable harm. | Hygiene (Eol software alone, permissive CSP alone, stack traces, info disclosure without concrete impact, "best practice" violations) |
19 | ==red_team== (external client engagement) | Hygiene findings + recon + IoCs + defensive-state observations are ALL deliverables | Nothing - even "no finding here" is reportable as a positive defensive observation |
20 | ==penetration (signed SOW / WAPT) | Depends on SOW. Read scope explicitly. Usually accepts hygiene + impact + recon | Out-of-scope assets, unsigned testing |
21 | ==internal_audit== | Compliance-mapped findings (PCI / ISO / NIST / DORA / GDS) | Findings without a control-mapping |
22 |
23 ==Hard rule== Before Phase 0 runs, write the engagement type as the first line in your hunt notes. If you can't answer it from the user's instruction, ASK once. Don't assume - the mistake costs both you and the triggerer.
24 |
25 ==Lesson from an authorized engagement== First-pass on this target produced 5 hygiene findings (SP2013 Eol, permissive CSP, stack traces) shipped in red-team format. The engagement was bug-bounty. Findings would have been N/A'd as "Informational, no impact demonstrated." After the corrected pass with hygiene-as-context-not-finding, the same target yielded 11 impact-demonstrated bugs including 3 Critical.
26 |
27 |
28 |
29 ## PART 1: MINDSET (How to Think)
30 |
31 ## Core Principle
32 |
33 Hunting is not "find a bug" - it is "prove an attack scenario." Think like an attacker with a specific goal, not a scanner looking for patterns.
34 |
35 ## Daily Discipline: Define, Select, Execute
36 |
37 Before touching any tool:
38 |
39 1. ==define== "Today I target [feature/domain] to achieve [CIA impact]"
40 2. ==select== Choose 1-2 vuln classes (IDOR, Race Condition, etc.)
41 3. ==execute== Focus ONLY on selected techniques. No wandering.
42 |
43 ## 5 Ultimate Goals (Pick One Per Session)
44 |
45 1. ==Confidentiality== - steal data the attacker shouldn't see
46 2. ==Integrity== - modify data the attacker shouldn't change
47 3. ==Availability== - disrupt service (app-level DoS only)
48 4. ==Account Takeover== - control another user's account
49 5. ==RCE== - execute commands on the server
50 |
  
```

A Skill can contain the title, description, methodology, tools, workflows, payloads, scripts and other resources that Claude can use while executing the task.

```
> /hunt

/hunt Active vulnerability hunting. Two-track dispatcher - asks Red Team vs WAPT, hands off to hunt-dispatch skill and sibling commands.
Usage: /hunt target.com | /hunt *.target.com | /hunt targets.txt [--vuln-class X] [--source-code P] [--chrome] (user)

/hunt-oto Hunt account takeover taxonomy - 9 distinct paths to AT0, plus chains. Paths: (1) password reset flows (host-header injection
redirects token, predictable/numeric token, Referer leak, no-expiry/reuse), (2) email change without re-auth, (3) OAuth account-li...

/hunt-rce Hunt skill for rce vulnerabilities. Built from 67 public bug bounty reports. Use when hunting rce on any target. (user)

/hunt-dom Hunt client-side DOM vulnerabilities - DOM Clobbering (overwrite JS globals via HTML injection), PostMessage hijacking (missing
origin check), Service Worker abuse (intercept requests from same-origin script), CSS Injection/Exfiltration (attribute selectors -

/hunt-lfi Hunt Local File Inclusion (LFI), Remote File Inclusion (RFI), and Path Traversal - /etc/passwd read, log poisoning + RCE, PHP
filter-chain RCE (no upload needed), php:// / data:// / zip:// / phar:// wrappers, RFI via allow_url_include, directory traversal -

/hunt-k8s Hunt Kubernetes & Docker - API anonymous access, kubelet 10258 exec (SPDY/WebSocket, NOT plain POST) and the simpler /run
primitive, etcd 2379 unauth, dashboard skip-login, RBAC misconfig, secret/SA-token abuse, docker.sock host escape, runc/container-

/hunt-xss Hunt skill for xss vulnerabilities. Built from 174 public bug bounty reports. Use when hunting xss on any target. For markup
injection that reflects raw HTML but does NOT execute JavaScript (no <script>/event-handler execution), see hunt-hml-injection -

/hunt-xxe Hunt skill for xxe vulnerabilities. Built from 18 public bug bounty reports including SVG-upload XXE, Office/doc (PP1X/DOCK)
XXE, SOAP XXE, SAML AssertionConsumer XXE, blind OOB XXE via OTD callback, parameter-entity XXE, XXE-to-LFI, XXE-to-SSRF, and XXE-

/hunt-saml Hunt SAML / SSO attacks. Patterns: XML Signature Wrapping (XSW) - modify Assertion while keeping Signature valid by relocating
signed element, comment injection in NameID (admin@target.com) - evil-->@attacker.com - some parsers see admin@target.com), signat...

/hunt-ssrf Hunt skill for ssrf vulnerabilities. Built from 15 public bug bounty reports including AWS metadata SSRF (HackerOne $25k
Analytics PDF, Shopify Exchange $25k, Capital One 106M-record breach, Dropbox/HelloSign $4,913), GCP metadata SSRF (Snapshot $4k),-

/hunt-sqli Hunt skill for sqli vulnerabilities. Built from 12 public bug bounty reports including modern NoSQL injection (Rocket.Chat
CVE-2021-22911 MongoDB $regex, Mongoose ORM CVE-2024-53900 $where bypass), modern ORM raw-frogment SQLi (Django CVE-2024-42805, Se...
```

Then, instead of giving Claude a long prompt every time, you can simply provide the Skill name and the target information. Claude can follow the workflow defined inside the Skill and generate the results in a consistent format.

Example Prompts for Creating Custom Bug Hunting Skills

Here are some examples of prompts you can use to turn your existing methodologies into reusable Skills.

Open Redirect Skill

Create a Claude Code Skill for finding and validating open redirect vulnerabilities.

Use the methodology below as the source material:

[PASTE YOUR OPEN REDIRECT METHODOLOGY HERE]

Convert this methodology into a complete reusable Skill. Include:

- Skill name and description
- Detection methodology
- Testing workflow
- Common parameters to test
- Payloads and bypass techniques
- Validation steps
- Useful CLI tools
- Expected findings and false positives
- Reporting workflow

Organize everything into a practical workflow Claude can follow during an authorized bug bounty assessment.

Do not invent techniques that are not supported by the methodology. Keep the Skill focused on open redirect testing.

XSS Skill

Create a Claude Code Skill for XSS hunting based on the methodology below:

[PASTE YOUR XSS RESEARCH OR NOTES HERE]

Turn this into a structured Skill that Claude can use during authorized security testing.

Include:

- Reconnaissance and input discovery
- Reflection detection
- Parameter and endpoint testing
- Context identification
- Payload selection
- Filter and encoding analysis
- Safe validation techniques
- Useful tools and commands
- False-positive handling
- Finding documentation and reporting

Preserve the methodology from my research and organize it into a repeatable workflow.

Custom Methodology → Skill

You can also give Claude an entire methodology and let it structure the Skill for you:

I have developed the following bug hunting methodology:

[PASTE YOUR COMPLETE METHODOLOGY HERE]

Convert this into a production-ready Claude Code Skill.

First understand the methodology and break it into logical phases.

Then create:

1. Skill name
2. Description
3. SKILL.md
4. Required tools
5. Testing workflow
6. Payloads or test cases
7. Helper scripts if required
8. Validation methodology
9. Reporting format
10. Any supporting resource files

Keep the workflow faithful to my original methodology.

The Skill will be used only for authorized security testing and bug bounty programs.

My locally installed security tools are available at:

[PASTE YOUR TOOL PATHS HERE]

Use these existing binaries when appropriate instead of attempting to install alternative versions.

You can then take the generated Skill, review the SKILL.md and any supporting files, make adjustments and place it in your Claude Skills directory.

This is where custom Skills become particularly useful. Instead of repeatedly explaining your methodology to Claude, you turn your experience into a reusable workflow that you can continuously improve over time.

You can also watch this video where I showed the complete practice of this method:

I

Conclusion

This workflow can save hours of repetitive work across traffic analysis, attack-surface mapping, testing and reporting. With Skills, you can also turn your own bug-hunting methodologies into reusable workflows and use open-source Skills from the community. The real power comes from combining MCP, Skills, strong prompts and your own methodology while still manually validating findings.

Disclaimer

The content provided in this article is for educational and informational purposes only. Always ensure you have proper authorization before conducting security assessments. Use this information responsibly.